

ARM PrimeCell

External Bus Interface (PL220)

Integration Manual

ARM PrimeCell External Bus Interface (EBI) Integration Manual

© Copyright ARM Limited 2002. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell External Bus Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	2
4.1	Supported Design Flow	3
4.2	Signal Description	4
4.3	Intra-chip connection	7
4.4	Primary input/output connection	7
4.5	Clocks	7
4.6	Resets	7
4.7	Scan Signals	8
4.8	Synchronisation	8
5	SYNTHESIS	8
5.1	Constraints	8
5.1.1	Intra-chip signals	8
5.1.2	Primary input/ output signals	8
5.1.3	Clocks	8
5.1.4	Resets	9
5.1.5	Scan signals	9
5.2	Synthesis scripts	9
5.2.1	Setup Scripts	9
5.2.2	Command Script	9
5.2.3	Constraint Scripts	10
6	PRODUCTION TEST	10
6.1	Scan Insertion and ATPG	10

7	FUNCTIONAL AND TIMING VERIFICATION	11
7.1	Functional verification	11
7.1.1	Formal Verification	11
7.1.2	Dynamic simulation	11
7.2	Timing verification	11
7.2.1	Static Timing analysis	11
7.2.2	Dynamic simulation	12
8	CUSTOMISATION	12
8.1	Changing bus width of address bus	12
8.1.1	Gate count change	12
8.1.2	RTL updates required	12
8.1.3	Time required for the change and difficulty level:	12
8.2	Changing bus width of data bus	12
8.2.1	Gate count reduction	13
8.2.2	RTL updates required	13
8.2.3	Time required for the change and difficulty level:	13
9	APPENDIX A	13
9.1	PrimeCell Environment Variables	13

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A01	15 th May, 2002	First draft
A02	1 Dec. 2002	Review comments incorporated

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0249A	ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual
3	PL220 DDES 0000	ARM PrimeCell External Bus Interface (PL220) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
EBI	External Bus Interface

2 SCOPE

This document describes how the ARM PrimeCell External Bus Interface (EBI) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell External bus Interface is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules has been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual [Ref.2].

The ARM PrimeCell External Bus Interface (PL220) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell External Bus Interface block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

4 INTEGRATION

The External Bus Interface (EBI) block is designed to be integrated into a system with up to three memory controllers or TIC sharing the external memory bus through a request-grant mechanism. The EBI does not have any AHB interface and does not directly connect to the AMBA. It provides a single channel for the memory controllers to access the off-chip memory devices. The EBI does not provide a fixed priority to the memory controllers connected to its three ports.

Figure 4-1 shows the integration of the EBI in an example system

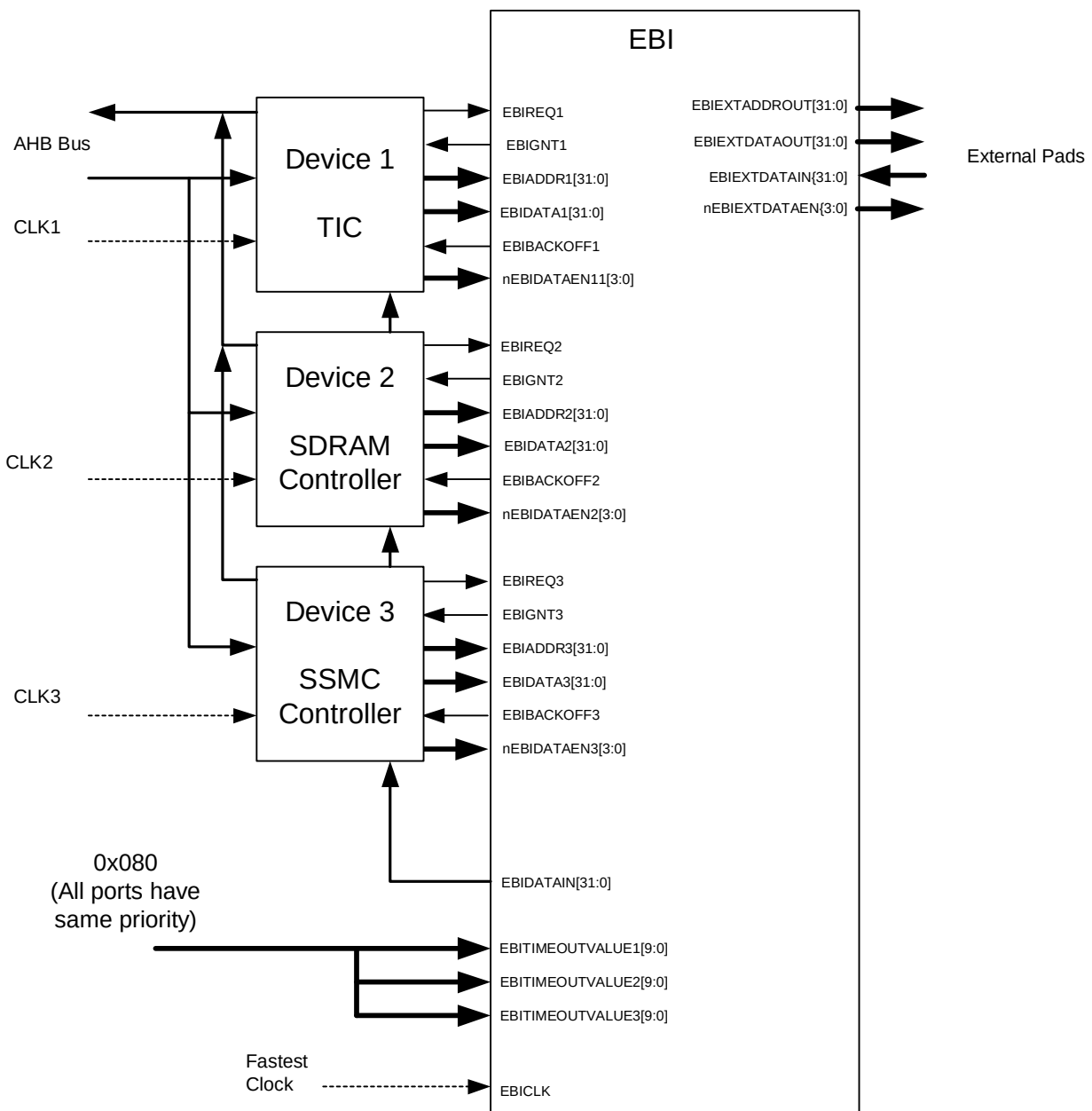


Figure 4-1 Example system showing connection of 3 devices to the external pads through the EBI

4.1 Supported Design Flow

The ARM PrimeCell External Bus Interface has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell External Bus Interface block.

An example ASIC design flow is described in **Figure 4.1-1**:

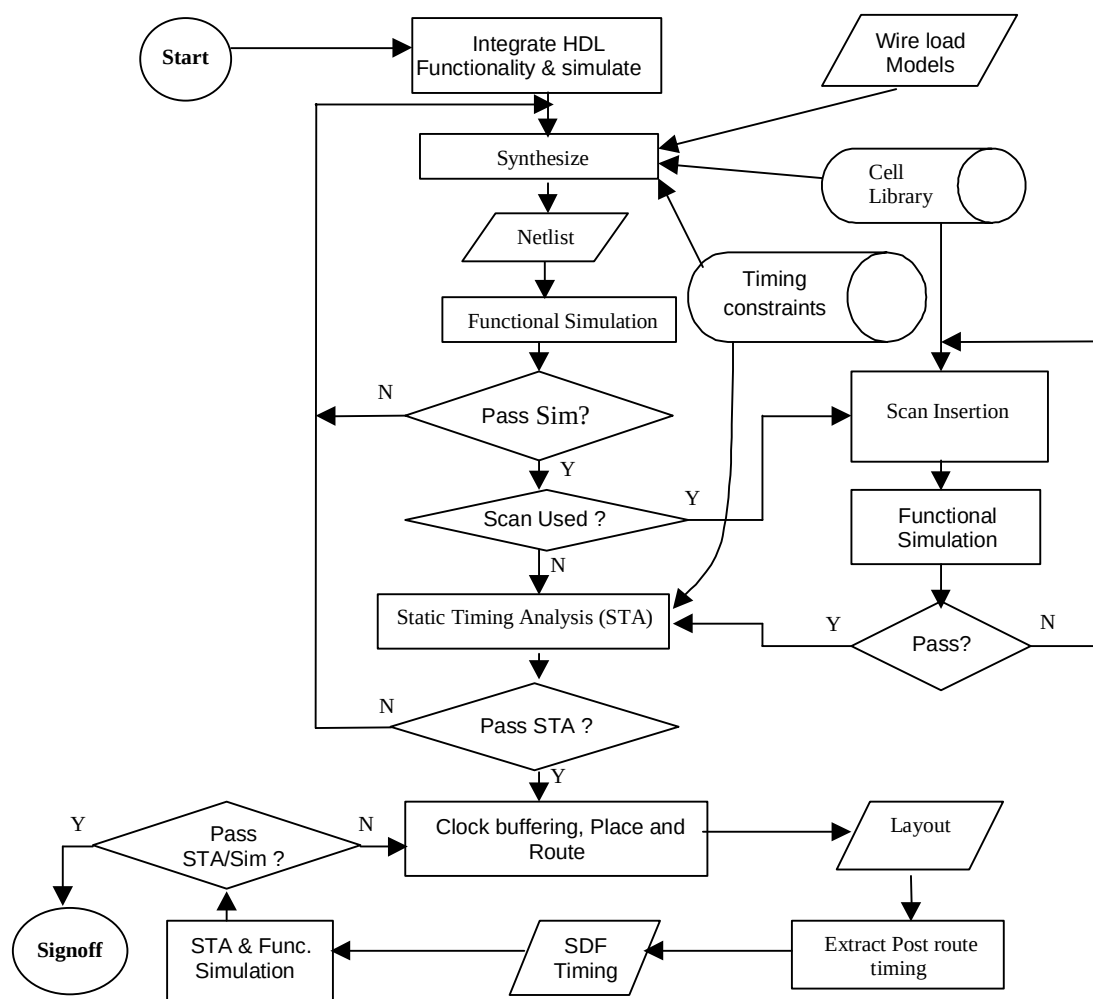


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Description

The following tables describe the EBI interface signals. **Tables 4.2-1** lists the clock signals in the EBI. **Tables 4.2-2** and **Table 4.2-3** list the memory controller / TIC side signals and Pad interface signals. **Tables 4.2-4** lists the scan test related signals in the EBI. For further information refer to the ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
EBICLK Memory clock	Input	Clock Source	External Bus Interface Clock. EBICLK must be the fastest clock among the memory controllers connected to the EBI. The EBI supports the use of memory controllers using different frequency clocks, but these must be integer divisions of the fastest clock. All the clocks must be synchronous and the fastest clock should be connected to the EBICLK

Table 4.2-1 EBI clock signals

Signal Name	Type	Source/Destination	Description
nPOR	Input	Reset Controller	Power on reset (active low).
EBIREQ1	Input	From the Controller 1	EBI request for port 1. Active high
EBIGNT1	Output	To the Controller 1	EBI grant for port 1. Active high
EBIBACKOFF1	Output	To the Controller 1	EBI backoff signal for port 1. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR1[31:0]	Input	From the Controller 1	EBI address for port 1.
EBIDATA1[31:0]	Input	From the Controller 1	EBI data for port 1
nEBIDATAEN1[3:0]	Input	From the Controller 1	Data enable for port 1.
EBITIMEOUTVALUE1[9:0]	Input	System Controller	Gives the value to be loaded into the timeout counter for port 1. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ2	Input	From the Controller 2	EBI request for port 2. Active high
EBIGNT2	Output	To the Controller 2	EBI grant for port 2. Active high
EBIBACKOFF2	Output	To the Controller 2	EBI backoff signal for port 2. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR2[31:0]	Input	From the Controller 2	EBI address for port 2.
EBIDATA2[31:0]	Input	From the Controller 2	EBI data for port 2
nEBIDATAEN2[3:0]	Input	From the Controller 2	Data enable for port 2.
EBITIMEOUTVALUE2[9:0]	Input	System Controller	Gives the value to be loaded into the timeout counter for port 2. Can be tied off

Signal Name	Type	Source/Destination	Description
			to a fixed value or supplied by the system controller.
EBIREQ3	Input	From the Controller 3	EBI request for port 3. Active high
EBIGNT3	Output	To the Controller 3	EBI grant for port the 3. Active high
EBIBACKOFF3	Output	To the Controller 3	EBI backoff signal for port 3. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR3[31:0]	Input	From the Controller 3	EBI address for port 3.
EBIDATA3[31:0]	Input	From the Controller 3	EBI data for port 3
nEBIDATAEN3[3:0]	Input	From the Controller 3	Data enable for port 3.
EBITIMEOUTVALUE3[9:0]	Input	System Controller	Gives the value to be loaded into the timeout counter for port 3. Can be tied off to a fixed value or supplied by the system controller.
EBIDATAIN[31:0]	Output	To all three Controllers	Data input connected to all the memory controllers

Table 4.2-2 Memory Controller / TIC side signals

Signal Name	Type	Source / Destination	Description
EBIEXTADDRROUT[31:0]	Output	Pad	EBI External Address Output. It is the multiplexed address bus connected to the memory
EBIEXTDATAOUT[31:0]	Output	Pad	EBI External data Output. It is the multiplexed data bus connected to the memory
nEBIEXTDATAENOUT	Output	Pad	EBI External Data Enable Output. It is the multiplexed data enable bus connected to the memory
EBIEXTDATAIN	Input	Pad	EBI External Data Input. It is connected to the Data line from the memory.

Table 4.2-3 Pad Interface signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.

Signal Name	Type	Source / Destination	Description
SCANINEBICKLK	Input	Test Controller	EBICKLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTEBICKLK	Output	Test Controller	EBICKLK domain Scan data output for observing the flip-flop contents in scan test mode.

Table 4.2-4 Scan Test related signals

4.3 Intra-chip connection

Up to 3 devices can be connected to the EBI. The priority is based on timeout counter as well as the round robin arbitration algorithm. The EBITIMEOUTVALUEx[9:0] has to be connected with some tied up value. It is possible to change the priority of the 3 ports by changing the EBITIMEOUTVALUEx[9:0].

4.4 Primary input/output connection

The primary inputs and outputs of EBI are the memory controller / TIC signals and external memory bus signals. For memory controllers that support the EBI with a backoff mechanism, these signals should be connected to the corresponding pins of the memory controller / TIC and the external memory device.

Some of the memory controllers use a different protocol for connecting to the EBI.

There are 2 types:

1. Devices with *EBIREQ* and *EBIGNT* signals and no backoff mechanism.
2. Devices with *EBIREQ* and *EBIGNT* signals and no-backoff signal but de-granting where the *EBIGNT* signal is used to also signal a backoff.

To interface to Type 1 the signal *EBIBACKOFF* can be left unconnected on the EBI. The device will not backoff from the bus when requested but will continue until it completes the transfer. This could hold the bus for a large amount of time.

To interface to Type 2 devices the memory controller clock must be the same as the EBI clock. A new grant signal for the device should be generated using the following logic:

$\text{NewEBIGNT} = \text{EBIGNT} \ \& \ \sim\text{EBIBACKOFF}$

If the memory clock is not the same as EBI clock then the *EBIBACKOFF* signal should be ignored and the same method as for type 1 should be used.

4.5 Clocks

The ARM PrimeCell EBI HDL contains only one clock sources at the top level – *EBICKLK*.

Each memory controller device can be clocked by a different clock. These must be integer divisions of the fastest clock and they must be positive edge aligned clocks. The fastest clock should be connected to the *EBICKLK*.

Please refer to ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual [Ref 2] for more details on the clocks in the EBI

4.6 Resets

The EBI block has only one reset pin – *nPOR*. *nPOR* is the power on reset.

HRESETn and nPOR can be asserted asynchronously by the system reset controller, but they should be de-asserted synchronous to the rising edge of HCLK. The power-on reset signal from the reset controller must be connected to the EBI nPOR signal. The internal logic of the EBI is reset by nPOR.

4.7 Scan Signals

SCANENABLE is the scan-mode enable pin of the EBI. The various scan signals are described in **Table 4.2-4**.

4.8 Synchronisation

The EBI contains only one clock. Therefore the design does not require any synchronization logic.

5 SYNTHESIS

The External Bus Interface block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Ebi.cmd - synthesizes the External Bus Interface to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Ebi_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

5.1 Constraints

The Ebi logic (Ebi) is synthesized at 190 MHz. The constraints on the different inputs and outputs of the EBI are described in the following sub-sections.

5.1.1 Intra-chip signals

The Non-AMBA Intra-chip signals are synthesised at the following constraints:

- 30% input setup on all non-AMBA Intra-chip inputs.
- 40% output valid delay on all non-AMBA Intra-chip outputs.

5.1.2 Primary input/ output signals

All primary inputs and outputs are synthesised at the following constraints

- 30% input setup on all primary inputs.
- 40% output valid delay on all primary outputs.

5.1.3 Clocks

Many different design methodologies are available for clock tree insertion and synthesis. The default External Bus Interface synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools, which understand

placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

5.1.4 Resets

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block

5.1.5 Scan signals

False paths are set from all scan related inputs so that the tool should not try to optimize the scan paths.

5.2 Synthesis scripts

The synthesis scripts consist of the following three main categories:

5.2.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb18_v1.0.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb18_v1.0.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

5.2.2 Command Script

The master compile command script Ebi.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Ebi.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Ebi_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top-level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

5.2.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file equivalent to the library_avanti_cb18_v1.0.scr file.
- b) The EBI is compiled at 190 MHz. These timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- The minus skew is set at 2.5% of the clock period. This is subtracted from the clock period used for the synthesis
- The plus skew is set at 40% of minus skew. This is used for the plus uncertainty value.

The plus skew derives from best case condition and it is less than the minus skew which derives from worst case condition.

The timing parameters assume the following bus timing budgets :

- 30 % input setup on all input ports .
 - 40 % output valid delay on all outputs
- c) Timing constraints for EBI-specific signals are set via the Ebi.scr. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.
 - d) Timing exceptions are specified via the Ebi_exceptions.scr file. The EBI does not have any exception

6 PRODUCTION TEST

The External Bus Interface is designed to be production tested using scan-insertion and ATPG.

6.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The External Bus Interface design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used, the number of chip level scan chains required may vary. As delivered, the External Bus Interface is synthesised 1 scan-chain.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

7 FUNCTIONAL AND TIMING VERIFICATION

It is essential to verify the functionality and the timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC

7.1 Functional verification

Functional verification is used to test the functionality of the design. The netlist should also be checked for functional equivalence to the RTL at all the key stages of the design flow. The functional verification can be performed by one of the two methods specified below:

7.1.1 Formal Verification

Formal verification is used to compare RTL with RTL and RTL with netlist. The functionality of the netlist or RTL can be compared to that of the RTL by use of a logic equivalence-checking tool. (For example, Synopsys Formality or Chrysalis). Functionality of the netlist and RTL can be compared within a shorter period of time by this method when compared to gate-level simulations.

7.1.2 Dynamic simulation

Verification tests are run on the RTL and netlist to confirm functionality of the design. Gate-level simulations can be used to compare the netlist functionality to that of the RTL. The same test vectors that were used to verify the RTL are used to verify the netlist functionality. This is however a time consuming process.

7.2 Timing verification

It is essential to verify timing performance of the netlist against the RTL at all the key stages of the design flow. Use of timing verification tools is a useful check for human error in the manual definition of false paths. The following two methods can be used to perform timing verification:

7.2.1 Static Timing analysis

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis.

7.2.2 Dynamic simulation

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

8 CUSTOMISATION

The ARM PrimeCell External Bus Interface can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Synthesis will have to be rerun to ensure that timing is met. Information to support such modification is included in the ARM PrimeCell External Bus Interface (PL220) Design Manual [Ref.3].

The following sections describe some of the customisations that are possible in the EBI and estimates of the gate count reduction / addition, the time required and difficulty level of the change and the RTL updates required for each customisation. The gate counts that will be achieved are dependent on the target cell library used, and synthesis timing constraints and command options.

8.1 Changing bus width of address bus

The bus width of the EBIADDR1, EBIADDR2, EBIADDR3 and EBIEXTADDRROUT can be changed by changing the signal buswidths of each of these uniformly. This will also change the pin count of the EBI. In case the address buswidths of the memory controllers connected to the three ports are different, the width of the widest bus should be used in the signal buswidths of all the ports. At the input ports of the EBI, the unused input pins should be tied LOW.

8.1.1 Gate count change

Gate count will change by a small amount for this modification.

8.1.2 RTL updates required

Only the signal bus widths have to be changed. The design should be synthesized again with these changes. The test cases and test bench should be modified to indicate the new bus width.

8.1.3 Time required for the change and difficulty level:

Approximate estimate: Between 1 and 2 person days, assuming similar experience. Between 2 and 3 person days, assuming little experience of the block.

Difficulty level: Low

8.2 Changing bus width of data bus

The bus width of the EBIDATA1, EBIDATA2, EBIDATA3, EBIEXTDATAOUT, EBIDATAIN, EBIEXTDATAIN, nEBIDATAEN1, nEBIDATAEN2, nEBIDATAEN3 and nEBIEXTDATAEN can be changed by changing the signal buswidths of each of these uniformly. This will also change the pin count of the EBI. In case the data buswidths of the memory controllers connected to the three ports are different, the width of the widest bus should be used in the signal buswidths of all the ports. At the input ports of the EBI, the unused input pins should be tied LOW.

8.2.1 Gate count reduction

Gate count will change by a small amount for this modification.

8.2.2 RTL updates required

Only the signal bus widths have to be changed. The design should be synthesized again with these changes. The test cases and test bench should be modified to indicate the new bus width.

8.2.3 Time required for the change and difficulty level:

Approximate estimate: Between 1 and 2 person days, assuming similar experience. Between 2 and 3 person days, assuming little experience of the block.

Difficulty level: Low

9 APPENDIX A

9.1 PrimeCell Environment Variables

Simulation and Synthesis in the External Bus Interface world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

e.g. :

- **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable.

e.g.:

- **setenv PERIPH EBI**

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g. :

- **setenv SIMULATOR modelsim**
- **setenv SIMULATOR LDV**

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

e.g. :

- **setenv TEST_METH noscan**
- **setenv TEST_METH scanready**
- **setenv TEST_METH scaninsert**

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. :

- **setenv HDL_SOURCE vhdl**
- **setenv HDL_SOURCE verilog**

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g. :

- **setenv HDL_NETL vhdl**
- **setenv HDL_NETL verilog**

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. :

- **setenv TEST_ENV BUSTALK**
- **setenv TEST_ENV TICTALK**

LIB_VHDL

The location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

e.g. :

- **setenv LIB_VHDL /AVANT/cb18/v1.0/vital/cb18os120/zero**

DIRS_VERILOG

The location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

e.g. :

- **setenv DIRS_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero**

FILES_VERILOG

The location of the verilog UDP (User Defined Primitive) file is defined by the '**FILES_VERILOG**' variable.

e.g. :

- **setenv FILES_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero/mtb_verilog.v**

LIB_SYNOP

The location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

e.g.:

- **setenv LIB_SYNOP /AVANT/cb18/v1.0/synopsys/cb18os120/1999.05/models**

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected - AHB, ASB or APB. Since the EBI trickbox is an AHB peripheral, set this environment variable to 'AHB'.

e.g. :

- **setenv AMBA AHB**